

FICHE DE TD 3/3 • DIFFICILE

Informatique • 1re TI

Algorithmes de tri et de recherche

Module 1 : Algorithmique

Cette fiche propose des problèmes de synthèse exigeant une maîtrise rigoureuse des invariants de boucle, de l'analyse de complexité et de l'adaptation algorithmique.

EXERCICE 1 Analyse formelle du tri par insertion

Soit l'algorithme de tri par insertion sur un tableau $T[1..n]$ (indexation à partir de 1).

1. Écrire en pseudo-code normalisé (style BAC) l'algorithme complet `TriInsertion(T, n)`.
2. Invariant de boucle : Formuler précisément l'invariant de la boucle principale (sur i de 2 à n). Démontrer sa validité par récurrence (Initialisation, Conservation, Terminaison) pour prouver la correction de l'algorithme.
3. Analyser la complexité en temps (nombre de comparaisons $T[j] > x$ et d'affectations) dans le meilleur, le pire et le cas moyen. Justifier chaque borne asymptotique ($\theta(\cdot)$).
4. Modifier l'algorithme pour qu'il trie par ordre décroissant tout en restant stable. Expliquer en une phrase pourquoi la modification préserve la stabilité.

EXERCICE 2 Recherche dichotomique : préconditions, variantes et seuils d'efficacité

On considère un tableau $T[1..n]$ trié par ordre croissant (doublons autorisés).

1. Écrire la fonction itérative `Dichotomique(T, n, x)` renvoyant l'indice de x ou 0 si absent. Préciser la précondition nécessaire sur T .
2. Variante « première occurrence » : Adapter l'algorithme précédent pour qu'il renvoie l'indice de la première occurrence de x en cas de doublons, sans passer par une recherche linéaire après coup. Donner le pseudo-code modifié et justifier la correction de la condition d'arrêt.
3. Seuil de rentabilité : On compare deux stratégies pour rechercher une valeur dans un tableau non trié de taille n :
 - Stratégie A : Recherche séquentielle directe.
 - Stratégie B : Tri par insertion + Recherche dichotomique.

En supposant le pire cas pour le tri et la recherche, déterminer la plus petite taille n_0 à partir de laquelle la Stratégie B devient asymptotiquement plus rapide que la Stratégie A. Discuter la pertinence de ce seuil pour de petites tailles ($n < 50$) en tenant

compte des coûts constants réels (affectations vs comparaisons).

EXERCICE 3 Synthèse : Maintenance d'un tableau trié sous flux de données

Une application embarquée (mémoire limitée, pas d'allocation dynamique) doit gérer un ensemble de $N_{max} = 1000$ entiers. Les opérations sont : `Inserer(x)`, `Supprimer(x)`, `Rechercher(x)`, `Afficher()`. Le tableau doit rester trié en permanence.

1. Proposer une structure de données (tableau + variables annexes) et écrire l'algorithme `Inserer(x)` qui insère x à la bonne position par décalage. Gérer le cas tableau plein. Complexité en temps ?
2. Écrire `Supprimer(x)` qui supprime une seule occurrence de x (la première trouvée) par décalage. Gérer l'absence de x . Complexité ?
3. Comparer l'efficacité globale de cette approche « Tableau trié » vs « Tableau non trié + Tri à la demande (TriInsertion) avant chaque recherche » pour un scénario de 500 insertions suivies de 500 recherches. Quantifier le nombre total d'opérations élémentaires (comparaisons + affectations) dans les deux cas (ordres de grandeur).
4. Question ouverte : Sans changer de structure (tableau contigu), proposer une optimisation pour réduire le coût des décalages lors des insertions/suppressions successives (indice : gap buffer ou tableau à trous). Décrire le principe en 3 lignes maximum.